2.2 Liquid Staking 팰릿 분석

개요

Liquid Staking 팰릿은 Composable Finance의 핵심 DeFi 인프라로, 릴레이체인(Polkadot/Kusama)의 네이티브 자산을 스테이킹하면서 동시에 유동성을 확보할 수 있는 서비스를 제공합니다. 사용자는 DOT/KSM을 스테이킹하고 stDOT/stKSM과 같은 파생 토큰을 받아 DeFi 활동에 활용할 수 있습니다.

주요 구성 요소

2.2.1 팰릿 구조

Liquid Staking 팰릿은 다음과 같은 주요 모듈로 구성됩니다:

- **lib.rs**: 메인 팰릿 로직 및 extrinsic 함수들
- **types.rs**: 데이터 타입 및 구조체 정의
- **distribution.rs**: 다중 계정 자산 분배 전략
- **relayer/**: XCM을 통한 릴레이체인 통신 모듈

2.2.2 핵심 데이터 구조

StakingLedger

```
#[derive(PartialEq, Eq, Clone, Encode, Decode, RuntimeDebug, TypeInfo)]
pub struct StakingLedger<AccountId, Balance: HasCompact> {
pub stash: AccountId,
```

```
#[codec(compact)]
pub total: Balance,
#[codec(compact)]
pub active: Balance,
pub unlocking: Vec<UnlockChunk<Balance>>,
pub claimed_rewards: Vec<EraIndex>,
}
```

MatchingLedger

```
#[derive(Copy, Clone, Eq, PartialEq, Default, Encode, Decode, RuntimeDebug, Type
pub struct MatchingLedger<Balance> {
pub total_stake_amount: ReservableAmount<Balance>,
pub total_unstake_amount: ReservableAmount<Balance>,
}
```

UnlockChunk

```
#[derive(PartialEq, Eq, Clone, Encode, Decode, RuntimeDebug, TypeInfo)]
pub struct UnlockChunk<Balance: HasCompact> {
#[codec(compact)]
pub value: Balance,
#[codec(compact)]
pub era: EraIndex,
```

```
}
```

2.2.3 스토리지 구조

ExchangeRate

```
#[pallet::storage]
#[pallet::getter(fn exchange_rate)]
pub  type ExchangeRate<T: Config> = StorageValue<_, Rate, ValueQuery>;
```

- **목적**: 릴레이체인 자산과 파생 토큰 간의 교환 비율
- **타입**: `StorageValue<Rate>`
- **기본값**: 제네시스 설정에서 초기화

StakingLedgers

```
#[pallet::storage]
#[pallet::getter(fn staking_ledger)]
pub  type StakingLedgers<T: Config> = StorageMap<
_,
Blake2_128Concat,
DerivativeIndex,
StakingLedger<T::AccountId, BalanceOf<T>>,
OptionQuery,
```

```
>;
```

- **목적**: 각 파생 계정의 스테이킹 원장 정보
- **키**: `DerivativeIndex` (파생 계정 인덱스)
- **값**: `StakingLedger` 구조체

MatchingPool

```
#[pallet::storage]
#[pallet::getter(fn matching_pool)]
pub type MatchingPool<T: Config> = StorageValue<_, MatchingLedger<BalanceOf<T>>
```

- **목적**: 각 에라별 스테이킹/언스테이킹 요청 매칭
- **기능**: 최적화된 릴레이체인 작업을 위한 요청 집계

Unlockings

```
#[pallet::storage]
#[pallet::getter(fn unlockings)]
pub type Unlockings<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AccountId,
Vec<UnlockChunk<BalanceOf<T>>>,
OptionQuery,
```

```
>;
```

- **목적**: 사용자별 언본딩 요청 및 해제 시기 추적
- **키**: 사용자 계정 ID
- **값**: 언본딩 청크 벡터

CurrentEra

```
#[pallet::storage]
#[pallet::getter(fn current_era)]
pub type CurrentEra<T: Config> = StorageValue<_, EraIndex, ValueQuery>;
```

- **목적**: 현재 에라 인덱스 추적
- **용도**: 언본딩 완료 시기 계산

2.2.4 주요 Extrinsic 함수들

stake

```
#[pallet::call_index(0)]
#[pallet::weight(<T as Config>::WeightInfo::stake())]
#[transactional]
pub fn stake(
origin: OriginFor<T>,
#[pallet::compact] amount: BalanceOf<T>,
```

```
) -> DispatchResultWithPostInfo
```

- **목적**: 릴레이체인 자산을 스테이킹하고 파생 토큰 발행
- **프로세스**:

1. 최소 스테이킹 금액 검증
2. 리저브 수수료 계산 및 차감
3. XCM 수수료 차감
4. 교환 비율에 따른 파생 토큰 발행
5. 매칭 풀에 스테이킹 요청 추가

- **이벤트**: `Staked`

unstake

```
#[pallet::call_index(1)]
#[pallet::weight(<T as Config>::WeightInfo::unstake())]
#[transactional]
pub fn unstake(
origin: OriginFor<T>,
#[pallet::compact] liquid_amount: BalanceOf<T>,
unstake_provider: UnstakeProvider,
) -> DispatchResultWithPostInfo
```

- **목적**: 파생 토큰을 소각하고 언스테이킹 요청 생성
- **언스테이킹 제공자**:

- `RelayChain` : 릴레이체인을 통한 일반 언스테이킹
- `MatchingPool` : 빠른 언스테이킹 (매칭 풀 활용)
- **프로세스**:

1. 최소 언스테이킹 금액 검증
2. 파생 토큰 소각
3. 언본딩 청크 생성 (타겟 에라 설정)
4. 매칭 풀에 언스테이킹 요청 추가

- **이벤트**: `Unstaked`

claim_for

```
#[pallet::call_index(11)]
#[pallet::weight(<T as Config>::WeightInfo::claim_for())]
#[transactional]
pub fn claim_for(
origin: OriginFor<T>,
dest: <T::Lookup as StaticLookup>::Source,
) -> DispatchResultWithPostInfo
```

- **목적**: 언본딩 완료된 자산을 사용자에게 반환
- **조건**: 현재 에라가 타겟 에라에 도달
- **프로세스**:

1. 현재 에라와 언본딩 청크의 타겟 에라 비교
2. 완료된 언본딩 청크 합계 계산

3. 자산 전송 및 청크 제거

- **이벤트**: `ClaimedFor`

2.2.5 릴레이체인 연동 함수들

bond

```
#[pallet::call_index(4)]
#[pallet::weight(<T as Config>::WeightInfo::bond())]
#[transactional]
pub fn bond(
origin: OriginFor<T>,
derivative_index: DerivativeIndex,
#[pallet::compact] amount: BalanceOf<T>,
payee: RewardDestination<T::AccountId>,
) -> DispatchResult
```

- **목적**: XCM을 통해 릴레이체인에서 본딩 실행
- **매개변수**:
- `derivative_index` : 파생 계정 인덱스
- `amount` : 본딩할 금액
- `payee` : 리워드 수령 계정 설정

bond_extra

```
#[pallet::call_index(5)]
#[pallet::weight(<T as Config>::WeightInfo::bond_extra())]
#[transactional]
pub fn bond_extra(
origin: OriginFor<T>,
derivative_index: DerivativeIndex,
#[pallet::compact] amount: BalanceOf<T>,
) -> DispatchResult
```

- **목적**: 기존 본딩에 추가 자산 본딩

unbond

```
#[pallet::call_index(6)]
#[pallet::weight(<T as Config>::WeightInfo::unbond())]
#[transactional]
pub fn unbond(
origin: OriginFor<T>,
derivative_index: DerivativeIndex,
#[pallet::compact] amount: BalanceOf<T>,
) -> DispatchResult
```

- **목적**: 릴레이체인에서 언본딩 실행

rebond

```
#[pallet::call_index(7)]
#[pallet::weight(<T as Config>::WeightInfo::rebond())]
#[transactional]
pub fn rebond(
origin: OriginFor<T>,
derivative_index: DerivativeIndex,
#[pallet::compact] amount: BalanceOf<T>,
) -> DispatchResult
```

- **목적**: 언본딩 중인 자산을 다시 본딩

withdraw_unbonded

```
#[pallet::call_index(8)]
#[pallet::weight(<T as Config>::WeightInfo::withdraw_unbonded())]
#[transactional]
pub fn withdraw_unbonded(
origin: OriginFor<T>,
derivative_index: DerivativeIndex,
num_slashing_spans: u32,
) -> DispatchResult
```

- **목적**: 언본딩 완료된 자산을 파생 계정으로 인출

nominate

```
#[pallet::call_index(9)]

#[pallet::weight(<T as Config>::WeightInfo::nominate())]

#[transactional]

pub fn nominate(

origin: OriginFor<T>,

derivative_index: DerivativeIndex,

targets: Vec<T::AccountId>,

) -> DispatchResult
```

- **목적**: 밸리데이터 후보들에게 노미네이션

2.2.6 매칭 시스템

MatchingLedger 구현

매칭 시스템은 스테이킹과 언스테이킹 요청을 효율적으로 처리하기 위한 핵심 메커니즘입니다:

matching 함수

```
pub fn matching(

&self,

unbonding_amount: Balance,

) -> Result<(Balance, Balance, Balance), DispatchError>
```

- **반환값**: `(bond_amount, rebond_amount, unbond_amount)`
- **로직**:

1. 스테이킹 요청 > 언스테이킹 요청: 초과분을 본딩
2. 언스테이킹 요청 > 스테이킹 요청: 차액을 언본딩
3. 언본딩 중인 자산이 있으면 우선 리본딩

매칭 최적화 전략

```
use Ordering::*;
let (bond_amount, rebond_amount, unbond_amount) =
if matches!(total_free_stake_amount.cmp(&total_free_unstake_amount), Less | Equ
(Zero::zero(), Zero::zero(), total_free_unstake_amount - total_free_stake_amount
} else {
let amount = total_free_stake_amount - total_free_unstake_amount;
if amount < unbonding_amount {
(Zero::zero(), amount, Zero::zero())
} else {
(amount - unbonding_amount, unbonding_amount, Zero::zero())
}
};
```

2.2.7 분배 전략 (Distribution Strategy)

DistributionStrategy 트레이트

```
pub trait DistributionStrategy<Balance> {
```

```
fn get_bond_distributions(
bonded_amounts: Vec<(DerivativeIndex, Balance, Balance)>,
input: Balance,
cap: Balance,
min_nominator_bond: Balance,
) -> Vec<(DerivativeIndex, Balance)>;
fn get_unbond_distributions(
active_bonded_amounts: Vec<(DerivativeIndex, Balance)>,
input: Balance,
min_nominator_bond: Balance,
) -> Vec<(DerivativeIndex, Balance)>;
fn get_rebond_distributions(
unbonding_amounts: Vec<(DerivativeIndex, Balance)>,
input: Balance,
) -> Vec<(DerivativeIndex, Balance)>;
}
```

AverageDistribution 전략

```
pub struct AverageDistribution;
```

- **특징**: 모든 파생 계정에 균등 분배
- **장점**: 단순하고 공정한 분배
- **용도**: 기본 분배 전략

MaxMinDistribution 전략

```
pub struct MaxMinDistribution;
```

- **본딩**: 가장 적게 본딩된 계정부터 우선 분배
- **언본딩**: 가장 많이 본딩된 계정부터 우선 분배
- **리본딩**: 가장 많이 언본딩 중인 계정부터 우선 분배
- **장점**: 계정 간 밸런스 최적화

2.2.8 교환 비율 시스템

교환 비율 계산

```
fn staking_to_liquid(amount: BalanceOf<T>) -> Option<BalanceOf<T>> {
let exchange_rate = Self::exchange_rate();
exchange_rate.reciprocal()?.checked_mul_int(amount)
}

fn liquid_to_staking(amount: BalanceOf<T>) -> Option<BalanceOf<T>> {
let exchange_rate = Self::exchange_rate();
exchange_rate.checked_mul_int(amount)
}
```

교환 비율 업데이트

교환 비율은 다음 요소들을 고려하여 업데이트됩니다:

- 스테이킹 리워드 누적
- 슬래싱 손실
- 수수료 및 리저브

2.2.9 XCM 요청 관리

XcmRequest 열거형

```
#[derive(PartialEq, Eq, Clone, Encode, Decode, RuntimeDebug, TypeInfo)]
pub  enum XcmRequest<T: Config> {
Bond { index: DerivativeIndex, amount: BalanceOf<T> },
BondExtra { index: DerivativeIndex, amount: BalanceOf<T> },
Unbond { index: DerivativeIndex, amount: BalanceOf<T> },
Rebond { index: DerivativeIndex, amount: BalanceOf<T> },
WithdrawUnbonded { index: DerivativeIndex, num_slashing_spans: u32 },
Nominate { index: DerivativeIndex, targets: Vec<T::AccountId> },
}
```

notification_received

```
#[pallet::call_index(10)]
#[pallet::weight(<T as Config>::WeightInfo::notification_received())]
#[transactional]
```

```
pub fn notification_received(
origin: OriginFor<T>,
query_id: QueryId,
response: Response,
) -> DispatchResultWithPostInfo
```

- **목적**: XCM 요청의 응답 처리
- **기능**: 스테이킹 원장 업데이트 및 상태 동기화

2.2.10 에라 관리 시스템

에라 진행 관리

```
fn do_advance_era(offset: EraIndex) -> DispatchResult {
let current_era = Self::current_era();
let new_era = current_era.saturating_add(offset);
CurrentEra::<T>::put(new_era);
IsMatched::<T>::put(false);
EraStartBlock::<T>::put(current_block);
Self::deposit_event(Event::<T>::NewEra(new_era));
Ok(())
}
```

자동 매칭 실행

```
#[pallet::hooks]

impl<T: Config> Hooks<BlockNumberFor<T>> for Pallet<T> {

fn  on_initialize(block_number: BlockNumberFor<T>) -> Weight {

// 에라 업데이트 확인

// 매칭 실행 조건 확인

// 자동 매칭 실행

}

}
```

2.2.11 빠른 언스테이킹 (Fast Unstake)

FastUnstakeRequests

```
#[pallet::storage]

#[pallet::getter(fn fast_unstake_requests)]

pub  type FastUnstakeRequests<T: Config> = StorageMap<

_,

Blake2_128Concat,

T::AccountId,

BalanceOf<T>,

ValueQuery,

>;
```

빠른 언스테이킹 매칭

- **수수료**: `MatchingPoolFastUnstakeFee` 설정값
- **장점**: 언본딩 기간 없이 즉시 자산 회수
- **조건**: 매칭 풀에 충분한 유동성 존재

2.2.12 보안 및 검증

주요 검증 사항

1. **최소 금액 검증**:

- `MinStake` : 최소 스테이킹 금액
- `MinUnstake` : 최소 언스테이킹 금액
- `MinNominatorBond` : 최소 노미네이터 본딩 금액

2. **교환 비율 유효성**:

```
ensure!(exchange_rate > Rate::zero(), Error::<T>::InvalidExchangeRate);
```

3. **시장 한도 검증**:

```
fn ensure_market_cap(amount: BalanceOf<T>) -> DispatchResult {
let cap = Self::staking_ledger_cap();
let total_bonded = Self::get_total_bonded();
ensure!(total_bonded.saturating_add(amount) <= cap, Error::<T>::CapExceeded);
Ok(())
}
```

4. **원장 잠금 상태**:

```
ensure!(!Self::is_updated(derivative_index), Error::<T>::StakingLedgerLocked);
```

트랜잭션 안전성

모든 주요 함수는 `#[transactional]` 매크로를 사용하여 원자성을 보장합니다:

```
#[transactional]
pub fn stake(origin: OriginFor<T>, amount: BalanceOf<T>) -> DispatchResultWith
```

2.2.13 이벤트 시스템

주요 이벤트들

Staked

```
Staked(T::AccountId, BalanceOf<T>)
```

- 스테이킹 완료 시 발생

Unstaked

```
Unstaked(T::AccountId, BalanceOf<T>, BalanceOf<T>)
```

- 언스테이킹 요청 시 발생 (계정, 파생토큰량, 스테이킹량)

Matching

```
Matching(BalanceOf<T>, BalanceOf<T>, BalanceOf<T>)
```

- 매칭 실행 시 발생 (본딩량, 리본딩량, 언본딩량)

ExchangeRateUpdated

```
ExchangeRateUpdated(Rate)
```

- 교환 비율 업데이트 시 발생

NewEra

```
NewEra(EraIndex)
```

- 새로운 에라 시작 시 발생

2.2.14 성능 최적화

배치 처리

- 매칭 시스템을 통한 요청 집계
- 에라별 일괄 처리로 XCM 호출 최소화

스토리지 최적화

- 언본딩 청크 통합으로 스토리지 효율성 증대
- 파생 계정별 원장 분리로 병렬 처리 가능

가스 효율성

- 조건부 실행으로 불필요한 연산 방지
- 스토리지 읽기/쓰기 최적화

이 Liquid Staking 팰릿은 Composable Finance 생태계에서 릴레이체인 자산의 유동성을 확보하면서도 스테이킹 리워드를 획득할 수 있는 핵심 인프라를 제공합니다. XCM을 통한 크로스체인 통신과 정교한 매칭 시스템을 통해 효율적이고 안전한 liquid staking 서비스를 구현합니다.